TicketHub

User Stories

Komplet samling af user stories til TicketHub-miniprojektet

Afledt fra casebeskrivelse, kommunikationsmønstre og SAGA-design

Event Catalog

User stories relateret til oprettelse, vedligeholdelse og browsing af events.

US-01: Opret event	
User Story	*Som arrangør vil jeg oprette et event med navn, dato, sted og beskrivelse, så eventet kan gøres klar til publicering.*
Service	Event Catalog

Acceptance Criteria:

1. Arrangøren angiver navn (påkrævet), dato (skal være i fremtiden), sted (påkrævet) og beskrivelse.
2. Eventet oprettes med status "Draft" og er ikke synligt for kunder.
3. Eventet tildeles et unikt ID.
4. Hvis navn mangler eller dato er i fortiden, afvises oprettelsen med en forståelig fejlbesked.

US-02: Tilføj billetkategorier	
User Story	*Som arrangør vil jeg tilføje en eller flere billetkategorier (f.eks. "Standard", "VIP") med navn, pris og antal til et event, så der kan sælges forskellige billettyper med forskellige priser.*
Service	Event Catalog

Acceptance Criteria:

1. Arrangøren angiver kategorinavn (påkrævet), pris (≥ 0) og antal billetter (> 0) for hver kategori.
2. Der kan tilføjes flere kategorier til samme event.
3. Kategorier kan kun tilføjes til events der ikke er annulleret.
4. Hvis pris er negativ eller antal ≤ 0, afvises tilføjelsen med en fejlbesked.

US-03: Publicer event	
User Story	*Som arrangør vil jeg publicere et event, så det bliver synligt for kunder og billetter kan sælges.*
Service	Event Catalog

Acceptance Criteria:

1. Kun events med status "Draft" kan publiceres.
2. Eventet skal have mindst én billetkategori før det kan publiceres.
3. Eventets status ændres til "Published".
4. Der udsendes en EventCreated-event via pub/sub, så Inventory-servicen kan oprette lagerposter.
5. Hvis eventet ikke har kategorier, afvises publiceringen.

US-04: Browse events	
User Story	*Som kunde vil jeg browse og søge i tilgængelige events, så jeg kan finde events jeg er interesseret i.*
Service	Event Catalog

Acceptance Criteria:

1. Kunden kan se en liste over publicerede events (ikke Draft eller Cancelled).
2. Kunden kan søge på navn og filtrere på dato.
3. Hvert event vises med navn, dato, sted, beskrivelse, status og billetkategorier med priser.
4. Kaldet er synkront (REST) da det er et klassisk forespørgsel/svar-mønster.

US-05: Se event-detaljer	
User Story	*Som kunde vil jeg se detaljerede oplysninger om et specifikt event, så jeg kan vurdere om jeg vil købe billetter.*
Service	Event Catalog

Acceptance Criteria:

1. Kunden kan hente et enkelt event via dets ID.
2. Svaret indeholder alle stamdata samt billetkategorier med navn, pris og total kapacitet.
3. Hvis eventet ikke findes, returneres en 404-fejl.

US-06: Markering af næsten udsolgt	
User Story	*Som systemet (Event Catalog) vil jeg automatisk markere et event som "næsten udsolgt" når Inventory melder lav beholdning, så kunder kan se at der snart ikke er flere billetter.*
Service	Event Catalog

Acceptance Criteria:

1. Event Catalog lytter på InventoryLow-events fra Inventory via pub/sub.
2. Når eventet modtages, ændres eventets status til "AlmostSoldOut".
3. Kun events med status "Published" kan markeres.

Inventory

User stories relateret til lagerstyring af billetter.

US-07: Automatisk oprettelse af lagerposter	
User Story	*Som systemet (Inventory) vil jeg automatisk oprette lagerposter når et event publiceres, så der kan holdes styr på tilgængelige billetter per kategori.*
Service	Inventory

Acceptance Criteria:

1. Inventory lytter på EventCreated-events via Dapr pub/sub.
2. For hver billetkategori i eventet oprettes en TicketStock-entity med Available = TotalCapacity og Reserved = 0.
3. Hvis eventet allerede har lagerposter (idempotens), oprettes der ikke dubletter.

US-08: Reserver billetter	
User Story	*Som systemet (Ordering via SAGA) vil jeg reservere et antal billetter i en given kategori, så billetterne låses midlertidigt så de ikke kan sælges til andre under bestillingsprocessen.*
Service	Inventory

Acceptance Criteria:

1. Inventory modtager en reservationsforespørgsel med ordreId, eventId og en liste af (categoryId, antal).
2. For hver linje reduceres Available og Reserved øges med det ønskede antal.
3. Hvis der ikke er nok tilgængelige billetter, fejler kaldet og ingen billetter reserveres.
4. Kaldet er synkront (REST via Dapr service invocation) da SAGA-workflowet skal vente på resultatet.

US-09: Frigiv billetter (kompensation)	
User Story	*Som systemet (Ordering via SAGA) vil jeg frigive tidligere reserverede billetter, så billetterne igen bliver tilgængelige for andre kunder når en bestilling fejler.*
Service	Inventory

Acceptance Criteria:

1. Inventory modtager en frigivelsesforespørgsel med ordreId, eventId og en liste af (categoryId, antal).
2. For hver linje øges Available og Reserved reduceres.
3. Dette er en kompenserende transaktion i SAGA-mønstret.
4. Kaldet er synkront (REST) og skal være idempotent.

US-10: Se lagerstatus	
User Story	*Som kunde eller system vil jeg se aktuel lagerstatus for et events billetkategorier, så det er synligt hvor mange billetter der er tilgængelige.*
Service	Inventory

Acceptance Criteria:

1. Lagerstatus kan hentes per event via REST.
2. Svaret indeholder per kategori: kategorinavn, tilgængelige, reserverede og total kapacitet.
3. Forespørgslen bruger AsNoTracking() for optimal læseperformance (CQS).

US-11: Advisering ved lav beholdning	
User Story	*Som systemet (Inventory) vil jeg udsende en InventoryLow-event når en billetkategori er tæt på at være udsolgt, så Event Catalog kan informere kunder om at billetter snart er væk.*
Service	Inventory

Acceptance Criteria:

1. Efter en reservation kontrolleres om Available/TotalCapacity er under en tærskelværdi (f.eks. 10%).
2. Hvis tærsklen er nået, publiceres en InventoryLow-event via pub/sub.
3. Event Catalog lytter og markerer eventet som “næsten udsolgt”.

Ordering

User stories relateret til bestillingsprocessen og SAGA-orkestreringen.

US-12: Placér bestilling	
User Story	*Som kunde vil jeg bestille billetter til et event ved at angive eventId, min e-mail og ønskede billetkategorier med antal, så jeg kan købe billetter til det event jeg ønsker.*
Service	Ordering

Acceptance Criteria:

1. Kunden sender en bestilling med eventId, e-mail og en liste af (categoryId, antal).
2. Ordering slår priser op synkront i Event Catalog via Dapr service invocation.
3. En ordre oprettes med status "Created" og unikke ordrelinjer med pris fra Event Catalog.
4. SAGA-workflowet (PlaceOrderWorkflow) startes automatisk.
5. Kunden modtager en bekræftelse på at bestillingen er modtaget.

US-13: SAGA: Happy path (bestilling gennemføres)	
User Story	*Som systemet (Ordering workflow) vil jeg gennemføre hele bestillingsflowet når alt lykkes, så kunden får sine billetter og modtager en bekræftelse.*
Service	Ordering

Acceptance Criteria:

1. Trin 1: Workflowet kalder Inventory for at reservere billetter (ReserveTickets). Succes: fortsæt.
2. Trin 2: Workflowet kalder Payment for at gennemføre betaling (ProcessPayment). Succes: fortsæt.
3. Trin 3: Ordrens status sættes til "Confirmed".
4. En OrderConfirmed-event publiceres via pub/sub.
5. Notification-servicen lytter og sender bekræftelse til kunden.

US-14: SAGA: Billetter ikke tilgængelige	
User Story	*Som systemet (Ordering workflow) vil jeg håndtere at billetter ikke er tilgængelige, så ordren afvises korrekt uden at efterlade inkonsistent tilstand.*
Service	Ordering

Acceptance Criteria:

1. ReserveTickets fejler fordi der ikke er nok billetter.
2. Ordren sættes til "Cancelled" med årsag "Tickets not available".
3. Der udføres ingen kompensation (der er intet at rulle tilbage).
4. En OrderCancelled-event publiceres via pub/sub.

US-15: SAGA: Betaling fejler	
User Story	*Som systemet (Ordering workflow) vil jeg håndtere at betaling fejler efter billetter er reserveret, så reserverede billetter frigives og ordren annulleres korrekt.*
Service	Ordering

Acceptance Criteria:

1. ReserveTickets lykkes, men ProcessPayment fejler.
2. Kompensation: Workflowet kalder Inventory for at frigive billetter (ReleaseTickets).
3. Kompensation: Ordrens status sættes til "Cancelled" med årsag indeholdende betalingsfejlen.
4. En OrderCancelled-event publiceres via pub/sub.
5. Notification-servicen lytter og sender annulleringsbesked til kunden.

US-16: SAGA: Betaling timer ud	
User Story	*Som systemet (Ordering workflow) vil jeg håndtere at betaling timer ud, så systemet ikke hænger og billetterne frigives efter timeout.*
Service	Ordering

Acceptance Criteria:

1. ReserveTickets lykkes, men ProcessPayment svarer ikke inden tidsfristen.
2. Timeout behandles som fejl: samme kompensationsflow som US-15.
3. Reserverede billetter frigives og ordren annulleres.

US-17: Se ordrestatus	
User Story	*Som kunde vil jeg se status på min bestilling, så jeg kan følge med i om min bestilling er bekræftet eller annulleret.*
Service	Ordering

Acceptance Criteria:

1. Kunden kan hente en ordre via ordreId.
2. Svaret indeholder ordreId, eventId, e-mail, status, totalbeløb, oprettelsestidspunkt og ordrelinjer.
3. Ordrelinjer viser kategorinavn, antal og enhedspris.
4. Hvis ordren ikke findes, returneres en 404-fejl.

Payment

User stories relateret til betalingsprocessen.

US-18: Behandl betaling	
User Story	*Som systemet (Ordering via SAGA) vil jeg gennemføre en betaling for en bestilling, så kunden betaler for sine billetter.*
Service	Payment

Acceptance Criteria:

1. Payment modtager ordreId, beløb og kundens e-mail.
2. Betalingen simuleres med konfigurerbar forsinkelse (200–800 ms).
3. Resultatet er enten succes (med transaktions-ID) eller fejl (med fejlbesked).
4. Fejlraten er konfigurerbar via miljøvariabel eller appsettings ("Payment:FailureRate").
5. Kaldet er synkront (REST) da SAGA-workflowet skal vente på resultatet.

Notification

User stories relateret til kundenotifikationer.

US-19: Bekræftelse ved gennemført bestilling	
User Story	*Som kunde vil jeg modtage en bekræftelse når min bestilling er gennemført, så jeg ved at mine billetter er sikret.*
Service	Notification

Acceptance Criteria:

1. Notification lytter på OrderConfirmed-events via Dapr pub/sub.
2. Når eventet modtages, logges en bekræftelsesbesked med ordreId og kundens e-mail.
3. Notifikationen simuleres via konsol-output/log (ingen reel e-mail).
4. Servicen har ingen forretningslogik – den reagerer udelukkende på events.

US-20: Besked ved annulleret bestilling	
User Story	*Som kunde vil jeg modtage en besked når min bestilling er annulleret, så jeg ved at bestillingen ikke gik igennem og kan prøve igen.*
Service	Notification

Acceptance Criteria:

1. Notification lytter på OrderCancelled-events via Dapr pub/sub.
2. Når eventet modtages, logges en annulleringsbesked med ordreId, e-mail og årsag.
3. Servicen er ren event-drevet og udelukkende koblet via messaging.

UI (Blazor)

User stories relateret til Blazor-baseret brugergrænseflade.

US-21: Browse events i UI	
User Story	*Som kunde vil jeg kunne se en liste over tilgængelige events i en Blazor-webapplikation, så jeg kan browse udbuddet uden at bruge API-kald direkte.*
Service	UI (Blazor)

Acceptance Criteria:

1. Blazor-siden viser en liste af publicerede events med navn, dato og sted.
2. Data hentes via API Gateway (YARP) der router til Event Catalog.
3. Listen opdateres når siden indlæses.

US-22: Se event-detaljer i UI	
User Story	*Som kunde vil jeg kunne klikke på et event og se detaljer inkl. billetkategorier, priser og tilgængelighed, så jeg kan beslutte om jeg vil købe billetter.*
Service	UI (Blazor)

Acceptance Criteria:

1. Detaljesiden viser event-information og billetkategorier med pris og antal tilgængelige.
2. Tilgængelighed hentes fra Inventory via API Gateway.
3. Der er en knap til at gå videre til bestilling.

US-23: Placér bestilling via UI	
User Story	*Som kunde vil jeg kunne vælge billetkategorier, angive antal og placere en bestilling via Blazor UI, så jeg kan købe billetter uden at bruge Swagger/Postman.*
Service	UI (Blazor)

Acceptance Criteria:

1. Bestillingsformular med valg af kategori og antal.
2. Bestillingen sendes via API Gateway til Ordering-servicen.
3. Brugeren ser en bekræftelse eller fejlbesked efter SAGA-flowet.

US-25: Se ordrestatus i UI	
User Story	*Som kunde vil jeg kunne se status på mine bestillinger i Blazor UI, så jeg kan følge med i om bestillingen er bekræftet eller annulleret.*
Service	UI (Blazor)

Acceptance Criteria:

1. Ordrestatussiden viser ordrer med status (Created, Confirmed, Cancelled).
2. Data hentes fra Ordering via API Gateway.
3. Status opdateres ved genindlæsning af siden.

API Gateway (YARP)

User stories relateret til YARP-baseret API Gateway.

US-24: API Gateway routing	
User Story	*Som udvikler vil jeg have en YARP-baseret API Gateway der router HTTP-requests til de korrekte microservices, så UI kun skal kende ét endpoint.*
Service	API Gateway (YARP)

Acceptance Criteria:

1. YARP er konfigureret med routes til Event Catalog, Inventory og Ordering.
2. Gateway kører som et selvstændigt ASP.NET Core-projekt i Aspire-løsningen.
3. Alle API-kald fra Blazor UI går gennem Gateway.
4. Routing-konfiguration er defineret i appsettings.json eller programmatisk.

US-26: Sundhedstjek via Gateway	
User Story	*Som udvikler vil jeg have et sundhedstjek-endpoint på API Gateway, der viser status på alle bagvedliggende services, så jeg hurtigt kan se om systemet er sundt.*
Service	API Gateway (YARP)

Acceptance Criteria:

1. GET /health på Gateway returnerer aggregeret sundhedsstatus.
2. Hvert bagvedliggende service har et /health-endpoint.
3. Gateway viser grøn/rød status per service.

Oversigt: User Stories per Service og Uge

Nedenfor er en matrice der viser hvilke user stories der hører til hvilken service og uge.

ID	User Story	Service	Uge
US-01	Opret event	Event Catalog	1–2
US-02	Tilføj billetkategorier	Event Catalog	1–2
US-03	Publicer event	Event Catalog	2
US-04	Browse events	Event Catalog	2
US-05	Se event-detaljer	Event Catalog	2
US-06	Markering af næsten udsolgt	Event Catalog	6
US-07	Automatisk oprettelse af lagerposter	Inventory	2
US-08	Reserver billetter	Inventory	3–4
US-09	Frigiv billetter (kompensation)	Inventory	4
US-10	Se lagerstatus	Inventory	2–3
US-11	Advisering ved lav beholdning	Inventory	6
US-12	Placér bestilling	Ordering	3
US-13	SAGA: Happy path	Ordering	4
US-14	SAGA: Billetter ikke tilgængelige	Ordering	4
US-15	SAGA: Betaling fejler	Ordering	4
US-16	SAGA: Betaling timer ud	Ordering	4
US-17	Se ordrestatus	Ordering	3
US-18	Behandl betaling	Payment	4
US-19	Bekræftelse ved gennemført bestilling	Notification	5
US-20	Besked ved annulleret bestilling	Notification	5
US-21	Browse events i UI	UI (Blazor)	7
US-22	Se event-detaljer i UI	UI (Blazor)	7
US-23	Placér bestilling via UI	UI (Blazor)	7
US-24	API Gateway routing	API Gateway (YARP)	7
US-25	Se ordrestatus i UI	UI (Blazor)	7
US-26	Sundhedstjek via Gateway	API Gateway (YARP)	7